Concept Drift Detection on NYC Taxi Demand Streams

Project plan for team review - Big Data mini project, Semester 5

Team	Rahul Saran Sharma, Rahul Prasanna, Raghav U
Date	27 October 2026
Submission	Option A - GitHub repository
Status	Draft. Please review and reply with approval or changes before any code is written.

1. Goal

A model that forecasts taxi demand gets worse as city travel patterns change. We will stream NYC taxi data through a Kafka + Spark pipeline and forecast next-hour pickups per zone. Streaming drift detectors (ADWIN, PSI and others) will watch the model. We will measure how fast they detect drift and how often they raise false alarms. Drift alarms will trigger automated retraining, and we will check whether drift hurts some boroughs more than others.

The course requires two components, each backed by **measured evidence**, not description alone:

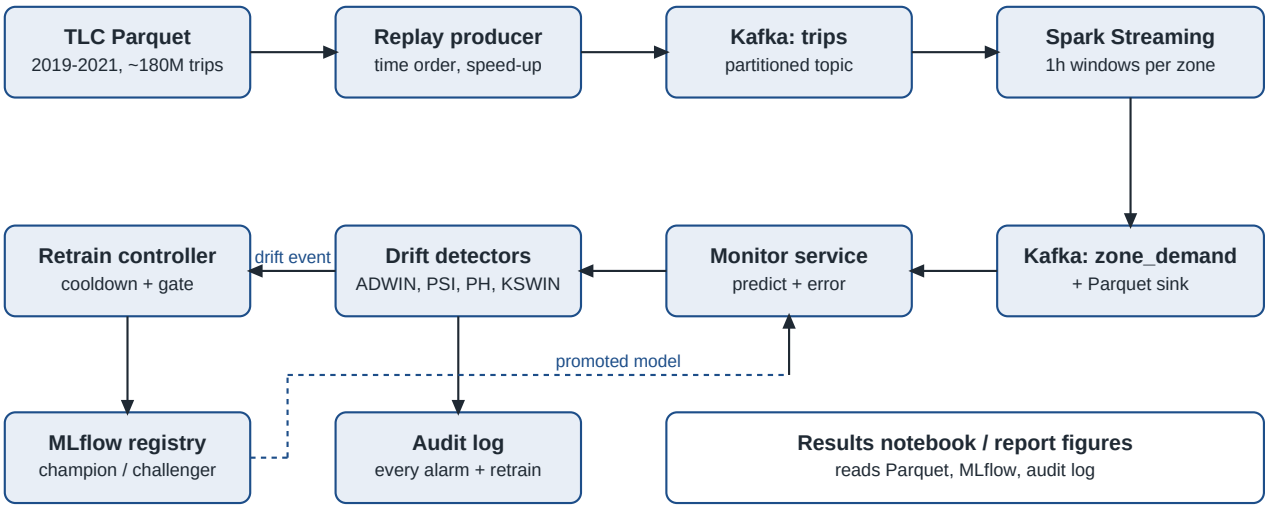
- **Big data systems**: Kafka streaming, distributed Spark processing, partitioned Parquet storage, with throughput, latency and scaling measurements.
- **Responsible AI**: fairness across boroughs, transparency (model and data cards), governance (an audit log of every retraining decision), privacy (zone-level aggregates only) and safety (a validation gate before a new model goes live).

2. Data

- **NYC TLC Yellow Taxi trip records**, monthly Parquet files, January 2019 to December 2021 (about 180 million trips, about 5 GB), plus the taxi zone lookup table (265 zones in 6 boroughs, including EWR). Public data; we only use pickup time and pickup zone, so nothing identifies a person.
- **Unit of prediction**: number of pickups per zone per hour.
- **Real drift with known dates**: COVID-19 collapse from about 15 March 2020 and NY PAUSE on 22 March 2020; phased reopening June-July 2020; Omicron in December 2021; holidays and seasons.
- **Reference period**: 2019 is stable. We inject synthetic drift into it so we know exactly when each drift starts (ground truth).
- **Optional**: public Census (ACS) median household income per zone, used only as an extra equity lens on errors.

3. Architecture

Everything runs from one `docker-compose.yml` on a laptop (Windows with Docker Desktop and WSL2). Services: kafka (single broker, KRaft mode), spark-master and spark-worker (scalable), producer, aggregator (Spark job), monitor, mlflow, jupyter.



- The **producer** replays trips in time order with a speed-up factor, so three years of data can stream in hours.
- **Spark Structured Streaming** groups trips into one-hour windows per zone, with a watermark for late events. It writes to a Kafka topic and to Parquet partitioned by year and month.
- The **monitor** predicts each hour, compares with the actual count, and feeds the errors and feature statistics to the detectors. Alarms go to the retraining controller and the audit log.

4. Method

4.1 Forecasting model

- LightGBM, one model for all zones. Features: demand lags (1, 2, 24 and 168 hours), rolling means, hour of day, day of week, holiday flag, zone id.
- Train on January-September 2019, validate on October-December 2019. Baseline to beat: same hour last week (seasonal naive).
- Metrics: MAE, and sMAPE, which is scale-free so busy and quiet boroughs can be compared.

4.2 Drift detectors (written by us, checked against the river library)

Detector	What it watches	Settings to compare
ADWIN	The model's error stream (catches drops in accuracy)	delta = 0.002, 0.01, 0.05
PSI	Distribution of demand and features in a sliding window vs the 2019 reference	168h window, 10 quantile bins, alarm at 0.1 (warning) and 0.25 (drift)
Page-Hinkley	Mean shift in the error stream	threshold and delta sweep
KSWIN	Kolmogorov-Smirnov test on a sliding window	window size, alpha

Each detector runs in two modes: **one city-wide detector** and **one detector per borough**. Manhattan has most of the trips, so a city-wide detector may not notice drift in Staten Island or the Bronx. Measuring this is part of the fairness analysis.

4.3 Synthetic drift benchmark

- Inject drift at known times into the 2019 stream: sudden demand change (x0.5, x1.5), gradual change over 1-4 weeks, rush-hour shift, recurring drift. Apply it to the whole city or to one borough only.
- At least 30 seeded runs per scenario, plus runs with no drift to count false alarms.

- Metrics: **detection delay** (hours from true change to first alarm), **false alarms per 1000 hours**, mean time between false alarms, missed-detection rate, delay per borough.
- Real-data check: how long after 15 March 2020 each detector fires.

4.4 Retraining policies

We replay 2019-2021 under each policy below and compare them:

Policy	When it retrains
Never retrain	Lower bound
Periodic	Every month, drift or not
ADWIN-triggered	When ADWIN fires on the error stream
PSI-triggered	When PSI goes above 0.25
Combined	PSI gives a warning, ADWIN confirms
Ideal baseline	At the known real change dates (upper bound, for comparison only)

- Controller rules: a cooldown (for example 72 hours) between retrains; train on a recent sliding window (4-8 weeks, configurable); the new model replaces the current one only if it is better on held-out recent hours; roll back if it gets worse.
- Compare: overall and per-borough MAE and sMAPE over time, number of retrains, total retraining time.
- Output: a written **retraining policy** (trigger, window, gate, cooldown, rollback) backed by these results.

5. Evidence we will measure

Component	Measurements
Big data systems	Spark streaming throughput (events per second) and end-to-end latency at different replay speeds; scaling with 1, 2 and 4 Spark workers and Kafka partitions; micro-batch processing time; raw vs aggregated storage size; query speed with Parquet partition pruning; time and memory per update for each detector.
Responsible AI: fairness	Per-borough error before and after drift; per-borough detection delay; gap between best and worst borough; whether each retraining policy narrows or widens the gap; city-wide vs per-borough detectors. Optional: error by zone income quartile.
Responsible AI: transparency and governance	Model card and data card; audit log of every alarm and retrain with reason and metrics; MLflow record of every model that went live.
Responsible AI: privacy and safety	Only zone-hour counts leave the aggregator; zones with very small counts noted in the data card; the validation gate and rollback stop a false alarm from putting a worse model live.

6. Team split (proposed)

Member	Owns	Main outputs
Rahul Saran Sharma	Infrastructure	Docker Compose, Kafka producer, Spark aggregator, Parquet storage, throughput and scaling experiments, README and architecture diagram
Rahul Prasanna	Drift detection	ADWIN, PSI, Page-Hinkley, KSWIN from scratch; tests against river; synthetic drift injector; detector comparison (delay, false alarms)
Raghav U	Model, retraining, fairness	Forecasting model; retraining controller and MLflow; retraining policy experiment; borough fairness analysis; model and data cards
All three	Shared	Results notebook, 3-5 page technical report, 3-minute demo video

7. Phases

Phase	Work	Done when
1. Setup and data	Download script, Spark batch aggregation of 2019, baseline model	Baseline accuracy on late 2019 is known
2. Pipeline and detectors	End-to-end stream on one month; detectors and unit tests	docker compose up produces drift events; tests pass
3. Detector experiments	Synthetic benchmark; real COVID replay	Delay and false-alarm tables for every detector
4. Retraining and fairness	All retraining policies; borough analysis; scaling runs	Policy comparison and fairness figures
5. Write-up	Notebook, report, cards, README, video	Every Option A item is in the repo

8. Submission checklist (Option A)

Required item	Where it will be
README with architecture diagram and setup steps	README.md
Reproducible pipeline	docker-compose.yml, Makefile, scripts/, experiments/
Results notebook	notebooks/results.ipynb (all figures from saved experiment outputs)
3-5 page technical report	docs/report.pdf
Model or data card	docs/model_card.md and docs/data_card.md
3-minute demo video	Link in README
Detector code, method comparison, retraining policy (topic brief)	src/drift/, experiments/run_detectors.py, src/retrain/policy.py and report section

9. How we will know it works

- All unit tests pass, and our detectors raise the same alarms as river on shared test streams.
- docker compose up on a clean machine runs the whole stream for one month and writes drift events and the audit log.
- One command reruns every experiment, and the notebook rebuilds every report figure from top to bottom.
- Sanity checks: ADWIN fires within days of 15 March 2020; runs on stable 2019 data give few false alarms; drift-triggered retraining has lower error than never retraining over 2020-2021.

Please reply with: approval, or changes to the scope, the team split or the phases.